DG1IAD, Internet Applications & Database Design

Aston University
BIRMINGHAM UK

Dr Alexandros Giagkos

Computer Science
College of Engineering and Physical Sciences

- **Week 1: Relational Databases and SQL**
  - **Lab: Interact with MySQL, and perform CRUD operations**
- Week 2: Functional Dependences and Normal Forms
  - Lab: Practising relationships and normalised designs
- Week 3: Data Integrity, ACID and Transactions
  - Lab: Complete unfinished lab work

Aston University
BIRMINGHAM UK

Relational Databases and SQL

Outline:

- Part I: From data to databases
- Part II: The Relational Model
- Part III: The Entity-Relationship Model
- Part IV: Introduction to SQL

Learning outcomes:

- Discuss database concepts, architectures and applications
- Understand the relational and entity-relationship models
- Introduce the structured query language (SQL)

From data to databases

What is data?

- Data are raw facts that on their own have no meaning
  - "the fact is a datum worth taking into account"

- Any alphanumerical character, i.e., text, numbers, symbols

- Example data sets:
  - Yes, No, No, Yes, Yes, Yes
  - 42, 65, 89, 143, 0
  - "CS1IAD", "CS1OOP", "CS1PSA"
  - , ,

Data to Information

- Data needs to be processed and turned into meaningful information
- It needs to be *structured* and *presented* in a given context

"Data" comes from a singular Latin word, datum, which originally meant **"something given"**. Its early usage dates back to the 1600s. Over time "data" has become the plural of datum.

"Information" is an older word that dates back to the 1300s and has Old French and Middle English origins. It has always referred to **"the act of informing"**, usually in regard to education, instruction or other knowledge communication.

Information to Knowledge

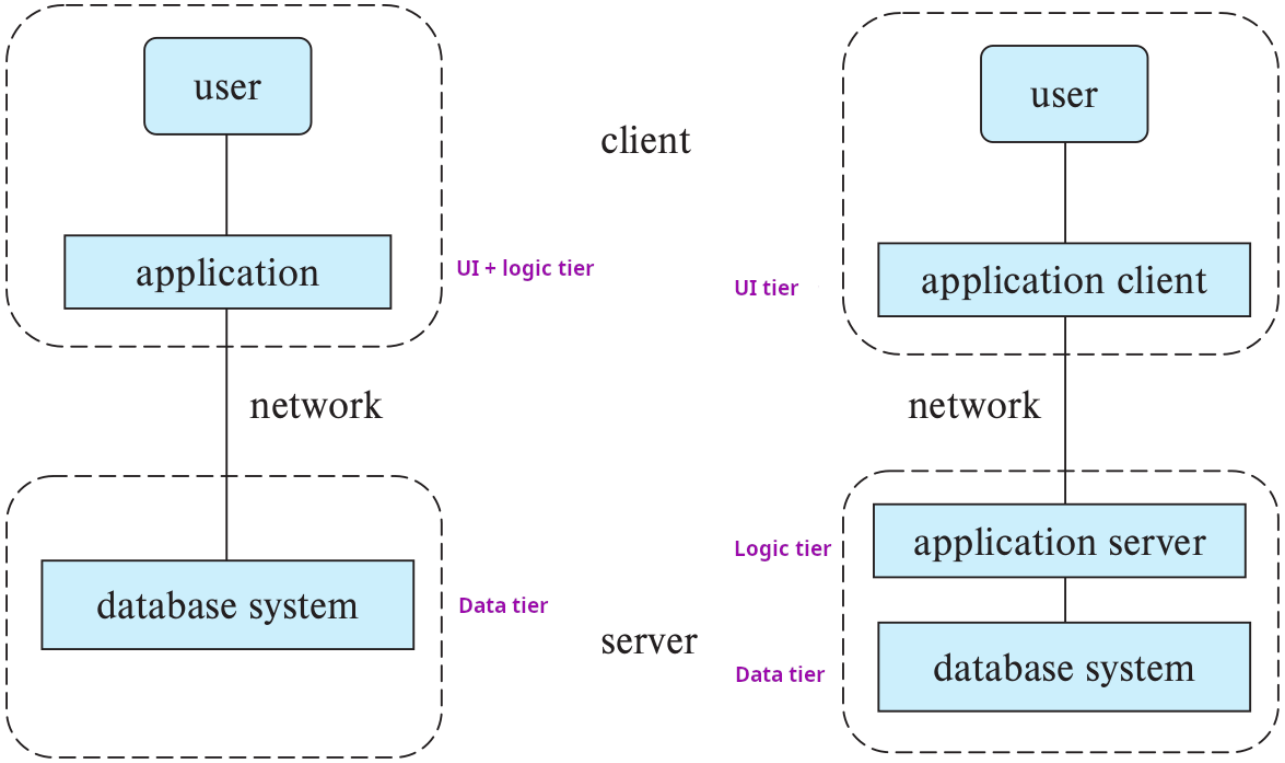
- Knowledge is the *understanding of rules* needed to interpret information
- The capacity of understanding relationships between pieces of information
- *What to actually do* with the information

Database Systems Applications

Aston University
BIRMINGHAM UK

- In everyday live, database system applications manage collections of data that are *highly valuable*, *relatively large* and *accessed by multiple* users and other applications
- Data, and not the calculations, is the central aspect
  - What is X or Facebook or Linkedin without data?

- Earlier-generation (a) versus modern-generation (b)



(a) Two-tier architecture (b) Three-tier architecture

- Note that tiers do not have to run on the same computer.

Storing data needs structure and rules

Aston University
BIRMINGHAM UK

A data model is a collection of *conceptual tools* that define the data structure, relationships, semantics and consistency constraints

We will look at:

- **Relational Model**: tabular format to accommodate data and to represent relationships
- **Entity-Relationship Model**: for database design using a collection of objects (entities) that have relationships with each other

Database Terminology

- A *database system* or *database management system (DBMS)* is a collection of interrelated data and a set of programs to access those data
- The collection of data is usually referred to as a *database*
- A *relational database* is based on the relational model
- The *SQL language* is used to interact with the database
  - Will be introduced later!
- The *database administrator (DBA)* has the central control over a database system

The Relational Model

Data Organisation in Relational Modelling

Aston University
BIRMINGHAM UK

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Instructor

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Course

course_id	*prereq_id*
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Prerequisites

Fundamentals of the Relational Model

Aston University
BIRMINGHAM UK

- Initial paper by Edgar Codd in 1970, "A Relational Model of Data for Large Shared Data Banks".
- Tables are used to structure the data related to a concept of reality, i.e., a university instructor
- Unique table columns describe variables related to the concept
- Rows represent the relationship among a list of values (*n-tuple of values*)
- In the relational model the term **relation** is used for a table, **tuple** for a row, and **attribute** for a column

Properties of Relations

- Relations of the same design have distinct names
- Each attribute has a distinct name
- Values of an attribute are all from the same domain
- Each cell in the relation contains exactly one atomic value
- Each tuple is distinct; there are no duplicate tuples
- No order for attributes or tuples
- Keys are used to uniquely identify tuples

Types of Keys

- Need to specify a method to distinguish tuples within a relation
- Attribute values can be used to uniquely identify a tuple
- A *superkey* is a set of one or more attributes which offers that uniqueness
  - e.g., {ID}, {ID, name}, {ID, dept_name}, etc., are all superkeys
- A *candidate key* is a minimal superkey for which no <u>proper subset</u> is a superkey
  - A relation may have multiple candidate keys, e.g., {ID} and {name, dept_name} are both candidate keys, whereas {ID, name} is not

Primary Keys in Relations

- The database designer decides the *primary key*
- Primary keys are referred to as *primary key constraints* and are usually listed first
  - In **relational schema notation**, e.g.:
    instructor (ID, name, dept_name, salary), or
    instructor (name, dept_name, ID, salary)
- Primary keys should be chosen wisely and w.r.t. what is modelled in the real-world
- A *surrogate key* may be used when a natural, or real, private key is not possible
  - e.g., {ID} with auto-increment

Foreign Keys in Relations

Aston University
BIRMINGHAM UK

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Instructor

DEPT_NAME	BUILDING	BUDGET	MONTHLY_ALLOW
CompSci	MB	1500000	125000
Maths	MB	1000000	83333.33
Physics	MB	1200000	100000
Geography	MB	1100000	91666.67
Mech.Eng.	MB	1100000	91666.67

department

- A *foreign key constraint* ensures that the value of an attribute of a relation matches the value of the primary key of some other relation
- e.g., {instructor.dept_name} is a *foreign key* referencing {department.dept_name}

Practical exercises

In relational databases, a _____ is a set of one or more attributes that uniquely identify tuples in a relation.

a) surrogate key

b) super key

c) foreign key

The Entity-Relationship Model

Entity Representation

- Entities represent distinctly identifiable items or "things"
  - e.g., a student, a university, a module
- They are related in a variety of ways
  - e.g., an instructor teaches a module

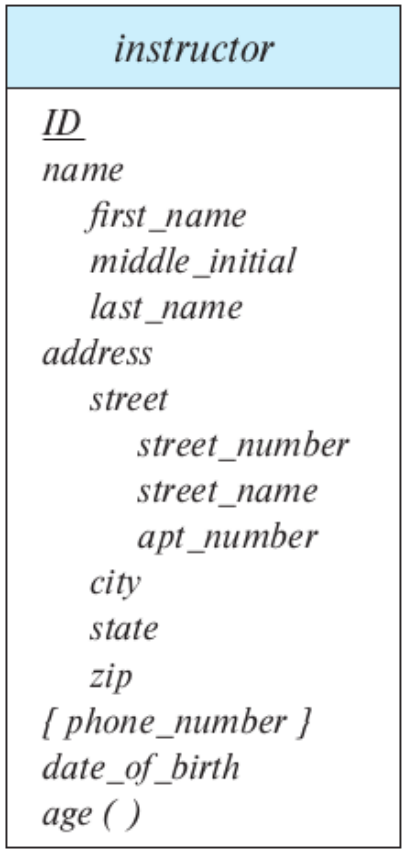
The Entity-Relationship Data Model

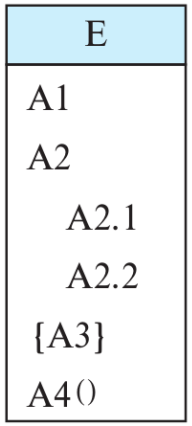
Aston University
BIRMINGHAM UK

- Original paper by Peter Chen in 1976. "The entity-relationship model–toward a unified view of data."
- The E-R model expresses the overall database design graphically, allowing conceptual database design and schema development
- Three main components: entities, attributes and relationships

Entities and Attributes in E-R Diagrams

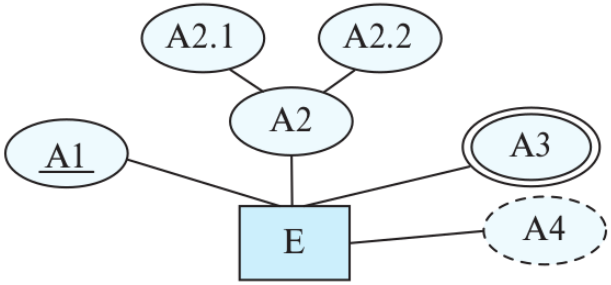
- Underlined primary key in both notations





attributes:
simple (A1),
composite (A2) and
multivalued (A3)
derived (A4)

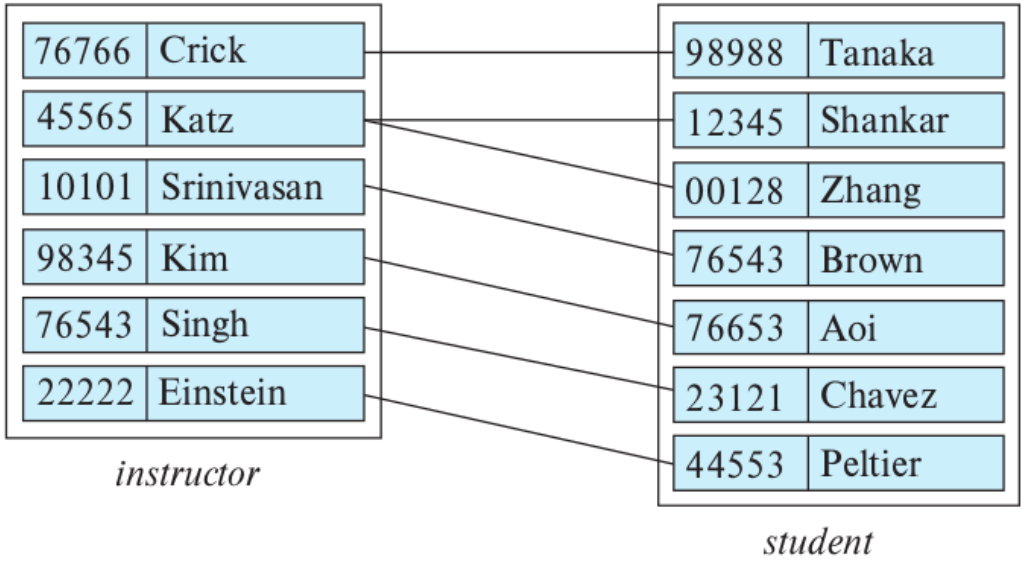
entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



Alternative notation

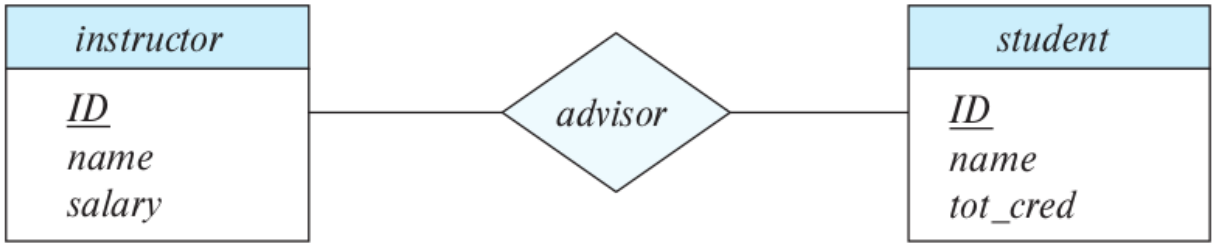
Relationships

- A relationship is an association among several entities
  - e.g., instructor Katz is associated with two students Shankar and Zhang



Relationship in E-R Diagrams

- A relationship is depicted by a diamond that links different related entities



- Entities participate in relationships, e.g., two entities participate in a binary relationship

Available types of relationships

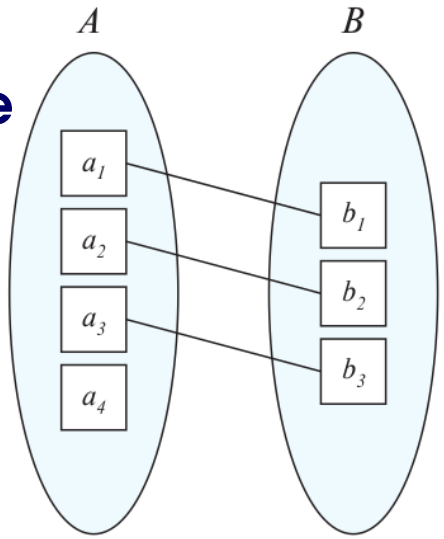
Aston University BIRMINGHAM UK

- Given the binary relationship between two entity sets A and B, there can be only one of the following mapping cardinalities:
  - **one-to-one:** an entity in A is associated with at most one entity in B, and vice versa
  - **one-to-many:** an entity in A is associated with any number of entities in B, whereas entities in B have one-to-one associations with entities in A
  - **many-to-one:** entities in A have one-to-one association with entities in B, whereas entities in B have one-to-many associations with entities in A
  - **many-to-many:** an entity in A is associated with any number of entities in B, and vice versa

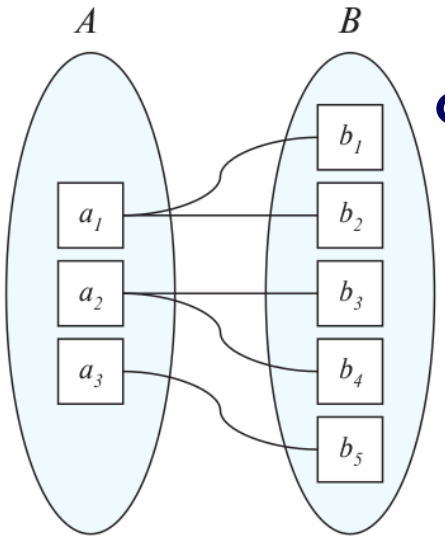
Available types of relationships (illustrated)

Aston University
BIRMINGHAM UK

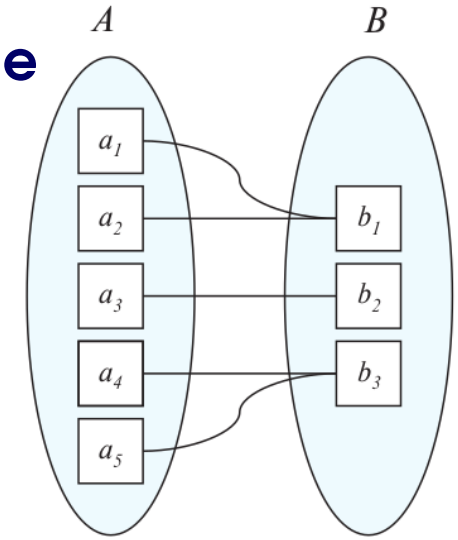
one-to-one



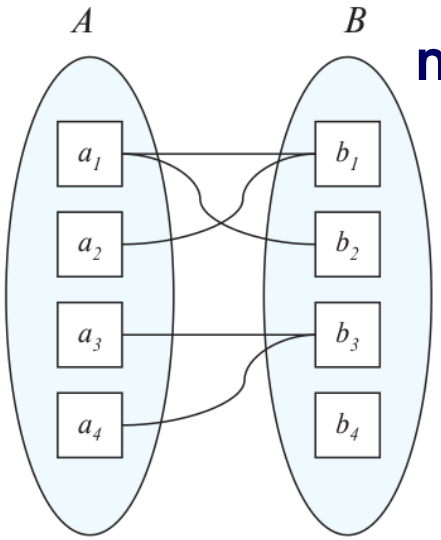
one-to-many



many-to-one



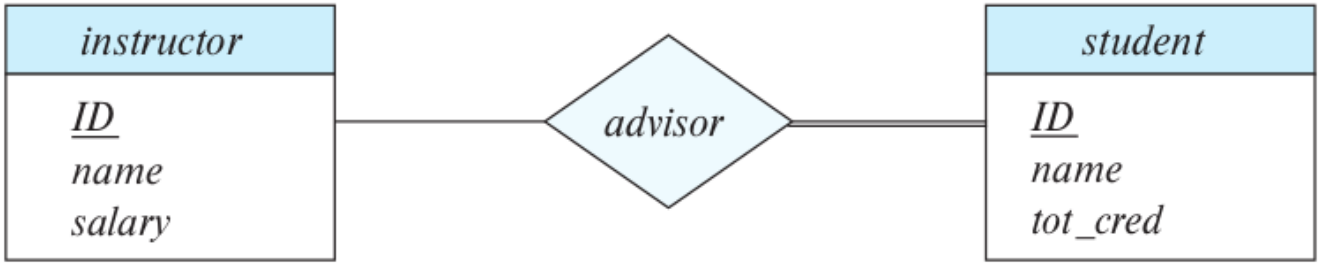
many-to-many



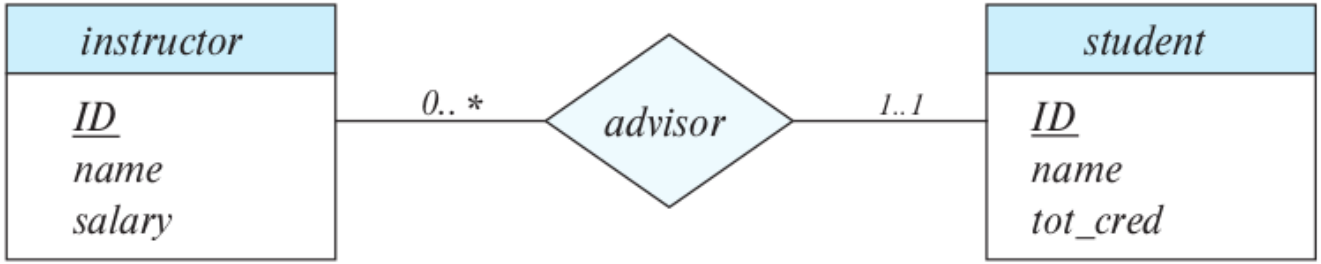
Mapping Cardinalities in E-R Diagrams

Aston University BIRMINGHAM UK

- The participation of an entity set A in a relationship is total, and depicted with double lines, when all entities in A participate in that relationship – partial otherwise



- Cardinality limits or multiplicities are used to depict complex relationships using the syntax of (min, MAX): e.g., (0..*), (0..1), (1..1), (1..*), etc.



Relationships to Relations (1/3)

- For one-to-one relationships, either side can be chosen to have an extra attribute, i.e., a foreign key referencing the primary key of the other relation.

  department (**dept_name**, building, budget)
  instructor (instructor_id, name, salary, **dept_name**)

  is similar to:

  department (dept_name, building, budget, **instructor_id**)
  instructor (**instructor_id**, name, salary)

- For partial participation on the "many" side, allow null values

Relationships to Relations (2/3)

- Similarly for one-to-many or many-to-one; now the "many" side is a given

  Many-to-one and one-to-many relationships can be represented by adding an extra attribute to the "many" side containing the primary key of the "one" side:

  department (**dept_name**, building, budget)
  instructor (instructor_id, name, salary, **dept_name**)

- For total participation do not allow null values

Relationships to Relations (3/3)

- A many-to-many relationship is represented by a new relation with attributes the primary keys of the two involved relations and any descriptive attributes of the relationship:

  department (**dept_name**, building, budget)
  instructor (**ID**, name, salary)

  we represent the relationship with a new relation:

  deptInstructor (**instructor_ID**, **dept_name**, allocation_date)

- It makes sense to name the new relation after the relations it joins together

- Note that the new relation may also have its own *descriptive attributes*, which refer to the relationship and not the entities per se

Introduction to SQL

Why using SQL?

Aston University
BIRMINGHAM UK

- It allows database management and manipulation tasks
- It is universal and portable and easy to learn
- It powers the most commonly used RDBMS
- It is the most popular language across desktop and web developers, system administrators, and data scientists

The important SQL parts

Aston University
BIRMINGHAM UK

- **Data Definition Language (DDL)**: provides commands for defining and manipulating relational databases, as well as specifying integrity constraints
- **Data Manipulation Language (DML)**: provides the ability to query the database, and to insert, delete and modify tuples in relations
- **Data Control Language (DCL)**: defines and manipulates access control of the database
- **Transaction Control Language (TCL)**: provides commands for dealing with transactions

SQL statements

- SQL statements may be multi-line but have to finish with a semicolon

```
CREATE TABLE department (
  dept_name VARCHAR(20) NOT NULL,
  building VARCHAR(150),
  budget DECIMAL(12,2),
  PRIMARY KEY (dept_name)
);
```

TABLE DEPARTMENT

Column	Null?	Type
DEPT_NAME	NOT NULL	VARCHAR2(20)
BUILDING	-	VARCHAR2(150)
BUDGET	-	NUMBER(12,2)

- All components are case insensitive, except for literal strings surrounded by single quotes

```
SeLeCT * FROM department
where dept_name = 'CompSci';
```

Create Relations

- Creating a department relation:

```
CREATE TABLE department (
  dept_name VARCHAR(20),
  building VARCHAR(15) NOT NULL,
  budget DECIMAL(12,2) NOT NULL,
  monthly_allow DOUBLE GENERATED ALWAYS AS (budget / 12) VIRTUAL,
  PRIMARY KEY (dept_name)
);
```

- Creating a course relation:

```
CREATE TABLE course (
  course_id     VARCHAR(7) ,
  title         VARCHAR(50) NOT NULL,
  dept_name     VARCHAR(20) NOT NULL,
  credits       INT DEFAULT 10,
  PRIMARY KEY (course_id),
  FOREIGN KEY (dept_name) REFERENCES department(dept_name)
);
```

Inserting Tuples

- Insert tuples by using the relation's definition order, e.g.,

```
CREATE TABLE department (
  dept_name VARCHAR(20),
  building VARCHAR(15) NOT NULL,
  budget DECIMAL(12,2) NOT NULL,
  monthly_allow DOUBLE GENERATED ALWAYS AS (budget / 12) VIRTUAL,
  PRIMARY KEY (dept_name)
);

INSERT INTO department VALUES ('Geography', 'MB', 1100000);
```

- Insert tuples by specifying the attributes and their order:

```
INSERT INTO department (building, dept_name, budget)
  VALUES ('MB', 'Geography', 1100000);
```

Updating Tuples

- "A salary increase of 2% is to be paid to all instructors with a salary less than 80K"

```
UPDATE instructor SET salary = salary * 1.02 WHERE salary < 80000;
```

- Updating multiple attributes is also possible:

```
UPDATE department SET building = 'AC', budget = 1600000
  WHERE salary < 80000;
```

Deleting Tuples

- "Delete all tuples of instructors, whose salary is between 85 and 100K"

```
DELETE FROM instructor WHERE salary BETWEEN 85000 AND 100000;
```

- Deletion will fail if any of the instructor's columns is referred by a foreign key of another relation

  Error: (...): integrity constraint (...) violated - child record found (...)

Dropping Relations

Aston University BIRMINGHAM UK

- Dropping a relation from the database is a drastic action:

```
DROP TABLE course;
```

- If referential constraints are in place, then DROP is cancelled:

```
DROP TABLE department;
```

e.g., if course exists, department cannot be deleted
Error: unique/primary keys in table referenced by foreign keys

- The following Oracle-specific clause will allow dropping referenced relations:

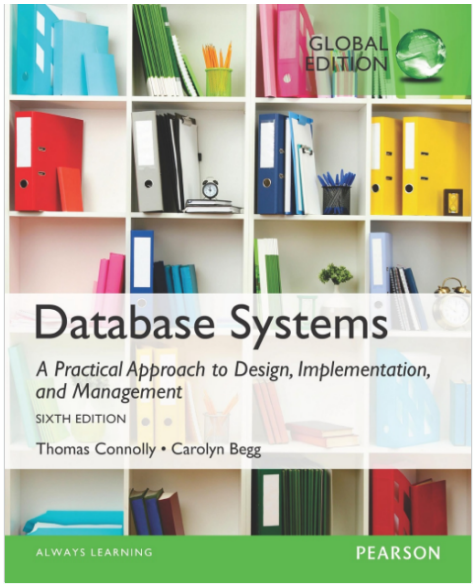
```
DROP TABLE department CASCADE CONSTRAINTS;
```

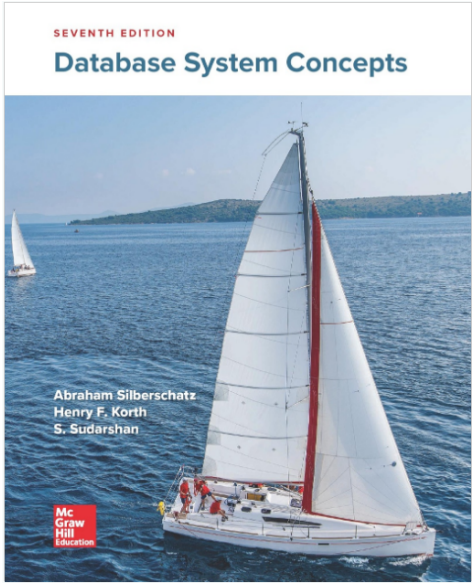
Explicitly declared (MySQL) when adding referential constraints in another relation:

```
FOREIGN KEY (dept_name) REFERENCES department(dept_name)
                                                ON DELETE CASCADE
```

Learning SQL

- SQL Tutorial: https://www.w3schools.com/sql/
- LiveSQL (Oracle): https://livesql.oracle.com/





Summary

You can now better discuss the following topics:

- Database systems architectures and applications
- The Relational and E-R Models
- How to use SQL statements to create, modify, populate and query a relational database